

چطور AI تست UI کتابخانه‌ها را از کار دستی به قرارداد خودکار تبدیل کرد.

خروجی کار، یک سیستم تست‌نویسی و کنترل کیفیت AI-assisted برای کتابخانه‌های مشترک Angular است: AI طبق قوانین پروژه spec تولید می‌کند، توسعه‌دهنده review می‌کند، Playwright و unit test قرارداد را enforce می‌کنند، و CI قبل از رسیدن تغییر به QA/Production باگ‌های ریز UI را متوقف می‌کند.

0

lib-level QA tickets in last cycle

1,000+

assertions per release / PR cycle

1d → <1h

time to create a full spec set

76

shared libraries under test

ایده اصلی و نیازی که مرتفع شد

CORE IDEA

قرارداد کتابخانه‌ها باید توسط ماشین enforce شود.

برای هر کتابخانه یک demo page واقعی داریم. AI بر اساس rule‌های پروژه unit spec و E2E spec می‌نویسد. Playwright همان صفحه واقعی را باز می‌کند و computed style، state، interaction و screenshot را چک می‌کند.

BEFORE

QA نقش firewall انسانی را داشت.

هر تغییر کوچک در یک package مثل ngx-button یا ngx-ngx spinner می‌توانست روی چند محصول اثر بگذارد. نتیجه: regression دستی، زمان زیاد، هزینه انسانی و ترس توسعه‌دهنده از refactor.

AI NEED

یکنواختی spec‌ها با rule‌های ثابت، نه prompt‌های پراکنده و خروجی‌های سلیقه‌ای.

ENGINEERING NEED

اعتماد به refactor برای shared package بدون اینکه هر تغییر تبدیل به release risk شود.

BUSINESS NEED

کاهش بار QA در سطح library regression و انتقال QA به user journey های مهم‌تر.

فیچرهای اصلی کاری که انجام شد

E2E LAYER

قرارداد **visual** و **interaction**

computed CSS, screenshot regression,
animation live-check, responsive,
.disabled/loading/destructive states

UNIT LAYER

قرارداد API کامپوننت

input/output, signal input, state class
و رفتار OnPush با coverage سخت‌گیرانه.

DEMO HARNESS

یک URL واقعی برای هر **library**

/package/<name> بدون mock و بدون
publish loop؛ تست مستقیم روی source و
همان چیزی که مصرف‌کننده می‌بیند.

CI GATE

merge قبل از QA متوقف می‌شود

اگر style, size, color, animation یا state
drift کند، PR در CI fail می‌شود.

LEARNING LOOP

هر **failure** تبدیل به **rule** شد

قانون جدید در TESTING.md و command
ثبت می‌شود و برای spec‌های بعدی از روز
صفر اعمال می‌شود.

AI RULES

قانون در **repo**، خروجی توسط AI

CLAUDE.md و slash-command
استاندارد تست را نگه می‌دارند؛ AI فقط
همان سبک را تولید می‌کند.

فلوی کاری از تغییر component تا CI سبز

05

CI gate

تست‌ها روی هر
push/PR اجرا می‌شوند
و regression را قبل از
merge می‌بندند.

04

بازبینی انسانی

Developer خروجی را
review می‌کند و edge
case‌ها را tune می‌کند.

03

تولید spec‌ها

Unit spec و
Playwright E2E spec
با همان الگوی پروژه
ساخته می‌شود.

02

اجرای AI command

با command
اختصاصی، AI سورس،
demo و rule‌ها را
می‌خواند.

01

توسعه component

Developer کامپوننت و
demo page را اضافه یا
اصلاح می‌کند.

IMPORTANT DETAIL

این flow جای developer را نمی‌گیرد؛ AI کار حجیم و تکراری تولید assertion‌ها را انجام می‌دهد، ولی rule‌ها، review، تصمیم نهایی و اصلاح edge case‌ها توسط تیم توسعه کنترل می‌شود.

نمونه بخش‌هایی که باید به چشم بیاید

Visual Regression

SCREENSHOT DIFF

RESULT

screenshots fail 24

AFTER



BEFORE



کشف باگ: تغییر رنگ brand primary در CSS global با چشم سخت
تشخیص داده می‌شود، اما screenshot regression همه variant‌ها را fail می‌کند.

ngx-spinner.spec.ts · Size Contract

CI BLOCK

```
const sizeSpec = {  
  sm: { size: '16px', stroke: '3px' },  
  md: { size: '20px', stroke: '4px' },  
  lg: { size: '24px', stroke: '4px' },  
};  
  
expect(actual.spinnerSize).toBe(expected.size);  
expect(actual.strokeWidth).toBe(expected.stroke);
```

کشف باگ: تغییر ناخواسته spinner-size - - از 16px به 15px در
review دیده نمی‌شود، اما Size Contract در CI fail می‌کند.

Input Variant Contract

STATE CHECK

کشف باگ: بعد از refactor، button در state disabled هنوز
pointer نشان می‌داد. تست DOM + style + screenshot را
همزمان چک کرد و merge را بست.

fullWidth

destructive

loading

disabled

Animation Contract

LIVE CHECK

```
const t1 = await spinner.evaluate(el =>  
  getComputedStyle(el).transform);  
await page.waitForTimeout(200);  
const t2 = await spinner.evaluate(el =>  
  getComputedStyle(el).transform);  
  
expect(t1).toBeTruthy();  
expect(t2).toBeTruthy();  
// t1 !== t2 → element is actually moving
```

نتیجه عملی برای تیم

Near 0

دستی در سطح regression بار library

7

spec: قرارداد ثابت در هر render, size, color, animation, visual, responsive, state

38

سبز برای پوشش E2E spec variantها و library

<1h

برای تولید تست کامل یک component با AI و review انسانی

FOR DEVELOPERS

- refactor shared packageها کم‌ریسک‌تر شد.
- هر تغییر قرارداد قابل اندازه‌گیری دارد، نه وابسته به حافظه افراد.
- AI خروجی اولیه حجیم را می‌سازد، developer فقط quality و edge case را کنترل می‌کند.

FOR QA

- QA از تست تکراری library plumbing آزاد شد.
- تمرکز QA به user journeyها، edge caseهای محصول و قضاوت انسانی منتقل شد.
- Regressionهای کوچک UI قبل از رسیدن به QA در CI پیدا می‌شوند.

چیزی که باید دیده شود: AI با ابزارهای واقعی کار می‌کند

FIGMA MCP RESULT

Design token ها تبدیل به rule دائمی شدند.

AI به جای استخراج rule، blind pixel value، design system را می‌شناسد: typography، spacing، color token و utility class ها.

MODEL CONTEXT PROTOCOL

Claude فقط متن نمی‌خواند؛ به ابزار وصل می‌شود.

با MCP می‌شود Figma، Sentry، GitHub، Google Drive یا API داخلی را داخل session در اختیار AI گذاشت. یعنی context واقعی، بدون copy/paste و بدون حدس.

with MCP rule

actual output

```
<h3 class="st-4 text-onback-high mb-medium">
  ...
</h3>
```

without rule

bad output

```
.title {
  font-size: 20px;
  color: #1f2937;
  margin-bottom: 16px;
}
```

ایجنت‌هایی که کار development را دقیق‌تر می‌کنند

[smart-dashboard-dev](#)

ایجنت پیاده‌سازی. برای یک task مشخص کار می‌کند، قبل از کدنویسی Figma/context را می‌خواند، اولویت را به component ها و package های موجود می‌دهد، از hardcode کردن style جلوگیری می‌کند، state های مختلف را پیاده می‌کند و قبل از تحویل build/lint/browser check انجام می‌دهد.

[smart-dashboard-figma-qa](#)

ایجنت QA سخت‌گیر. بعد از diff، implementation را بررسی می‌کند، rule های Angular/base-style/RTL را enforce می‌کند، Figma را با structured context و screenshot می‌سنجد، app واقعی را با browser باز می‌کند، و اگر چیزی ناقص باشد PASS نمی‌دهد.

[smart-dashboard-design-review](#)

ایجنت final visual gate. فقط visual fidelity را بررسی می‌کند: layout, spacing, color, typography, icon, RTL، فارسی، state ها و flow ها. معیارش «تقریباً شبیه» نیست؛ هر اختلاف باید با مقدار دقیق مثل px/class/hex گزارش شود.

OPERATING MODEL

نقش‌ها از هم جدا شده‌اند: یک agent می‌سازد، یکی QA می‌کند، یکی pixel-perfect review می‌کند. این جداسازی باعث می‌شود AI هم سرعت بدهد و هم کیفیت را با gate های مستقل بالا ببرد.